



가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 05 - Land Use and Building Growth Lab

Chris Ohk

utilForever@gmail.com

- Lecture 4의 토지 이용과 성장 규칙을 적용한다.
 - 용도지역과 건물을 구분하고, 입주와 전출을 계산한다.
- 7개의 함수를 작성하며 지난주에 만든 도시 모델을 확장한다.
 - 작성 → 단계 검사 → 화면 확인 → 조건을 바꿔 실험

- 압축을 푼 실습 폴더에서 실행한다.
- Windows: `py -m tkinter → py app.py`
- macOS / Linux: `python3 -m tkinter → python3 app.py`
- `model.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

- 제공된 코드에서 TODO 주석 부분을 채운다.

파일	역할	수업 중 수정
app.py / gui.py	실행 / 지도 / 버튼 표시	제공
city_data.py	초기 도시 / 편집 / 저장	제공
model.py	토지 이용 / 성장 규칙	직접 작성
checks.py	단계별 자동 검사	실행

- 기본 도시는 지난주와 같은 5 × 4 지도다.
- 주택 A는 6/10명, 주택 B는 8/10명, 대기 3명, 시작 일자는 0일이다.

화면 기능	조작 / 확인	연결되는 실습
용도지역 / 시설 배치	R 지정 후 주택 배치	TODO 1
선택한 셀	점유율 / 도로 / 환경 조회	TODO 2~5
입주 대기	도시 전체의 대기 인원	TODO 6
1일 진행	인구 / 대기 / 누적 전출	TODO 7

- 도시는 하나의 딕셔너리로 관리한다.
- grid는 위치, buildings는 객체, demand는 입주 대기 인원이다.

위치	필드	의미
셀	zone / building_id	용도 / 실제 건물 참조
주택	population / capacity	거주 인구 / 수용 인원
상업 / 공업	workers / capacity	점유 일자리 / 전체 일자리
도시	demand / departed	대기 인원 / 누적 전출

- 용도지역은 셀에, 실제 시설의 정보는 건물 객체에 저장한다.

```
cell = city["grid"][y][x]
zone = cell["zone"]
building_id = cell["building_id"]

if building_id is not None:
    building = city["buildings"][building_id]
```

실습 1 : 용도 적합성

- `can_build(city, x, y, kind)`를 작성한다.
 - 셀의 용도에 맞는 건물인지 `True / False`로 반환한다.
 - 주택은 R / 상업시설은 C / 공업시설은 I에 대응한다.
 - 공원은 용도를 지정하지 않은 셀에 배치한다.
- 좌표 / 지형 / 도로 / 기존 건물 검사는 제공한다.

- `model.py`의 TODO 1을 완성해 보자.
 - `python checks.py --stage 1`
 - R에 주택은 허용하고, 상업시설은 거부해야 한다.
 - R로 지정해도 건물을 배치하기 전에는 건물이 없다.
- 이 함수는 적합성만 판단하고 도시를 바꾸지 않는다.

- 건물 종류별로 필요한 용도를 정한 뒤 셀의 zone과 비교한다.

```
allowed_zone = {  
    "house": "R",  
    "shop": "C",  
    "factory": "I",  
    "park": None,  
}  
return kind in allowed_zone and cell["zone"] == allowed_zone[kind]
```

- occupancy(building)는 0~1의 비율을 반환한다.
 - 주택은 거주 인구 / 수용 인원으로 계산한다.
 - 상업시설 / 공업시설은 점유 일자리 / 전체 일자리로 계산한다.
- 공원이거나 수용량이 0이면 None을 반환한다.
 - 0.6을 60%로 출력하는 동작은 GUI에서 처리한다.

- `model.py`의 TODO 2를 완성해 보자.
 - `python checks.py --stage 2`
 - 6/10은 0.6, 0/10은 0.0을 반환한다.
 - 프로그램을 다시 실행하고 주택 A의 60%를 확인한다.
- 공원과 수용량 0인 건물은 무엇을 반환할까?

- 계산할 수 없는 경우를 먼저 처리하고 점유량을 수용량으로 나눈다.

```
def occupancy(building):  
    if building["kind"] == "park" or building["capacity"] == 0:  
        return None  
    if building["kind"] == "house":  
        quantity = building["population"]  
    else:  
        quantity = building["workers"]  
    return quantity / building["capacity"]
```

- `road_access(city, building)`를 작성한다.
 - 상하좌우 중 도로가 하나라도 있으면 `True`를 반환한다.
 - 도로가 없으면 `False`를 반환한다.
- 지난주에 작성한 `neighbors4()`를 재사용한다.
 - 중심 셀과 대각선은 확인 대상에서 제외한다.

이웃 셀에서 도로 찾기

- B(3, 1)의 아래쪽 (3, 2)에 도로가 있다.
 - 도로망 전체의 연결 여부는 이번 실습에서 다루지 않는다.

이웃 좌표	현재 상태	도로 여부
(2, 1)	공원 P	False
(4, 1)	물	False
(3, 0)	빈 땅 또는 공업시설	False
(3, 2)	도로	True

- `model.py`의 TODO 3을 완성해 보자.
 - `python checks.py --stage 3`
 - 기본 도시의 A / B는 모두 `True`다.
 - B 아래 도로 (3, 2)를 지우면 B는 `False`다.
- 첫 이웃이 도로가 아니면 곧바로 `False`를 반환해도 될까?

- 도로를 발견하면 True, 모든 이웃을 확인한 뒤에는 False를 반환한다.



```
def road_access(city, building):  
    for nx, ny in neighbors4(city, building["x"], building["y"]):  
        if city["grid"][ny][nx]["road"]:  
            return True  
    return False
```

- `environment_score(city, building)`는 E 를 반환한다.
 - $E = 2 \times \text{인접 공원 수} - 3 \times \text{인접 공업시설 수}$
 - 상하좌우의 운영 중인 시설만 센다.
- 이웃 셀의 `building_id`로 실제 객체를 찾는다.
 - 산업지역으로 지정한 빈 땅은 공업시설로 세지 않는다.

- 같은 공원에 인접해도 공업시설의 위치에 따라 E가 달라진다.

도시 / 주택	공원 P	공업시설 I	환경 점수 E
기본 / A	1개	0개	2
기본 / B	1개	0개	2
공업시설 인접 / A	1개	0개	2
공업시설 인접 / B	1개	1개	-1

- `model.py`의 TODO 4를 완성해 보자.
 - `python checks.py --stage 4`
 - 기본 도시에서는 A / B 모두 $E = 2$ 다.
 - 공업시설 인접 도시에서는 B의 $E = -1$ 이다.
- 빈 산업지역과 운영을 중지한 공업시설은 어떻게 처리할까?

- 실제 이웃 건물의 종류와 운영 상태를 확인하며 점수를 누적한다.

```
def environment_score(city, building):  
    score = 0  
    for nx, ny in neighbors4(city, building["x"], building["y"]):  
        ident = city["grid"][ny][nx]["building_id"]  
        if ident is None:  
            continue  
        neighbor = city["buildings"][ident]  
        if neighbor["status"] != "operating":  
            continue  
        if neighbor["kind"] == "park":  
            score += 2  
        elif neighbor["kind"] == "factory":  
            score -= 3  
    return score
```

- `growth_request(city, house)`를 작성한다.
 - (입주 요청 인원, 전출 인원)을 튜플로 반환한다.
 - 운영 중인 주택만 계산한다.
- 입주 요청은 빈자리, 전출은 현재 인구로 제한한다.
 - 실제 입주 인원은 다음 실습에서 배정한다.

- 도로 접근성과 환경 점수로 세 경우를 구분한다.

조건	입주 요청	전출
도로 없음 또는 $E \leq -1$	0	$\min(1, q)$
도로 있음 / $E = 0$	0	0
도로 있음 / $E \geq 1$	$\min(2, K - q)$	0

- `model.py`의 TODO 5를 완성해 보자.
 - `python checks.py --stage 5`
 - 기본 도시의 A는 (2, 0), 공업시설 인접 도시의 B는 (0, 1)이다.
 - 만실인 주택은 요청 0명, 인구 0인 주택은 전출 0명이다.
- 함수 호출 뒤에도 인구와 demand는 그대로여야 한다.

- 쇠퇴 조건을 먼저 확인한 뒤 유지와 성장을 구분한다.

```
def growth_request(city, house):  
    if house["kind"] != "house" or house["status"] != "operating":  
        return (0, 0)  
    score = environment_score(city, house)  
    if not road_access(city, house) or score <= -1:  
        return (0, min(1, house["population"]))  
    if score == 0:  
        return (0, 0)  
    vacancy = house["capacity"] - house["population"]  
    return (min(city["rules"]["growth_per_day"], vacancy), 0)
```

- `allocate_arrivals(requests, demand)`를 작성한다.
 - 주택별 요청과 대기 인원을 받아 실제 배정량을 반환한다.
 - 건물 ID를 문자열 오름차순으로 처리한다.
- 배정량의 합은 도시의 대기 인원을 넘을 수 없다.
 - 입력 `requests`는 바뀌지 않고 새 딕셔너리를 반환한다.

대기 인원 3명 나누기

- A와 B가 각각 2명을 요청해도 실제 입주는 총 3명이다.

순서	요청	실제 배정	남은 대기
시작			3명
A	2명	2명	1명
B	2명	1명	0명

- `model.py`의 TODO 6을 완성해 보자.
 - `python checks.py --stage 6`
 - A의 요청이 2명, B의 요청이 2명, 대기 3명이면 실제 배정은 A 2명, B 1명이다.
 - 대기가 0명이면 모든 주택의 배정량도 0명이다.
- 입력 딕셔너리에서 B가 먼저 나와도 결과가 같아야 한다.

- 남은 대기 인원을 하나만 관리하고 배정한 만큼 차감한다.



```
def allocate_arrivals(requests, demand):  
    allocated = {}  
    remaining = demand  
    for ident in sorted(requests):  
        count = min(requests[ident], remaining)  
        allocated[ident] = count  
        remaining -= count  
    return allocated
```

- `step(old)`는 다음 날의 새 도시 상태를 반환한다.
 - `old`에서 판단하고, 복사한 `new`에만 기록한다.
 - 요청과 전출 계산 / 수요 배정 / 상태 기록 순서로 진행한다.
- 인구 / 대기 인원 / 누적 전출 인원을 갱신한다.
 - 예산과 날짜 처리는 제공된 코드를 사용한다.

이전 상태와 다음 상태

- 모든 주택의 변화량은 같은 시작 상태에서 계산한다.
- 주택을 순회하면서 old의 인구를 바꾸지 않는다.
- 지역별 계산과 도시 전체 배정을 순서대로 연결한다.

단계	읽는 값	기록하는 곳
요청 / 전출 계산	old의 주택과 이웃	requests / departures
입주 배정	requests / old의 demand	allocated
다음 상태 기록	old / allocated / departures	new

- 주택별 결과를 합쳐 도시 전체 상태를 갱신한다.
 - 다음 인구 = 현재 인구 + 실제 입주 - 전출
 - 다음 대기 = 현재 대기 - 실제 입주 합계
 - 다음 누적 전출 = 현재 누적 전출 + 오늘 전출 합계
- 전출자는 대기 인원에 다시 더하지 않는다.
 - 요청 합계 대신 실제 배정 합계를 사용한다.

- `model.py`의 TODO 7을 완성해 보자.
 - `python checks.py --stage 7`
 - 기본 도시에서 1일 진행하면 건물 A 8명 / 건물 B 9명 / 대기 0명이다.
 - `step(old)`를 호출한 뒤에도 `old`의 0일 상태는 그대로여야 한다.

- 먼저 모든 요청과 전출을 모으고, 도시 수요를 한 번 배정한다.

```
def step(old):  
    validate_city(old)  
    new = deepcopy(old)  
    start_population = total_population(old)  
    requests = {}  
    departures = {}  
    for ident, house in old["buildings"].items():  
        if house["kind"] == "house":  
            requests[ident], departures[ident] = growth_request(old, house)  
    allocated = allocate_arrivals(requests, old["demand"])
```

- 각 주택의 새 인구와 도시의 합계를 new에 기록한다.



```
for ident in requests:
    new["buildings"][ident]["population"] = (
        old["buildings"][ident]["population"] + allocated[ident] - departures[ident]
    )
new["demand"] = old["demand"] - sum(allocated.values())
new["departed"] = old["departed"] + sum(departures.values())
```

하루 진행 결과 확인

- 기본 도시와 공업시설 인접 도시에서 결과를 확인한다.
- 공업시설 인접 도시는 A의 경우 $E = 2$, B의 경우 $E = -1$ 이다.

도시 / 날짜	A	B	인구	대기	누적 전출
기본 / 1일	8	9	17	0	0
공업시설 인접 / 0일	6	8	14	5	0
공업시설 인접 / 1일	8	7	15	3	1
공업시설 인접 / 2일	10	6	16	1	2
공업시설 인접 / 3일	10	5	15	1	3

비교 실험 : 공업시설 제거

- 공업시설만 삭제한 도시와 기준 도시를 비교한다.
- 각 경우에 공업시설 인접 도시를 다시 선택해 0일부터 시작한다.

조건 / 1일 결과	A	B	대기	누적 전출
공업시설 있음	8	7	3	1
공업시설만 삭제	8	10	1	0

- 공업시설 인접 도시를 다시 선택한다.
 - B 아래 도로 (3, 2)만 삭제하고 1일 진행한다.
- $A = 8 / B = 7 / \text{대기} = 3 / \text{누적 전출} = 10$ 이다.
 - 도로를 삭제하기 전과 같은 결과가 나오는 이유는 무엇일까?
- B는 이미 $E = -10$ 이므로 쇠퇴 조건을 만족하고 있었다.

- 기본 도시의 빈 땅 (0, 1)을 R로 지정한다.
 - 건물이 생겼는지 확인한 뒤 같은 셀에 주택을 배치한다.
- 새 주택은 인구 0명 / 수용량 10명이다.
 - 하루가 지나도 입주하지 않는 이유를 설명해 보자.
- 조회 화면에서 도로 접근성과 환경 점수를 확인한다.

- 용도 확인 → 입지 계산 → 수요 배정 → 상태 갱신
 - 7개의 함수를 통해 수요에 따른 건물의 성장과 쇠퇴를 구현했다.
- `python checks.py`로 전체 결과를 확인해 보자.

감사합니다.

utilForever@gmail.com

<https://github.com/utilForever>

Facebook, Twitter, Instagram, LinkedIn: utilForever